



Infraestrutura de Software Implementação 1

Atenção:

- Os arquivos de implementação deverão ser salvos dentro de um diretório cujo nome será formado pelas iniciais do e-mail em minúsculas. Exemplo: considerando que o e-mail é mla@cesar.school, o diretório será mla. Qualquer diretório enviado fora desse padrão será automaticamente descartado;
- O diretório deverá ser compactado em um arquivo .tar e disponibilizado para o(a) professor(a) via Google Classroom. O nome do arquivo deverá ser formado pelas iniciais do e-mail em minúsculas. Exemplo: considerando que o e-mail é mla@cesar.school, o arquivo a ser entregue será mla.tar. Qualquer arquivo enviado fora desse padrão será automaticamente descartado;
- Identificada a cópia de qualquer questão, seja em relação a outro(a) aluno(a) ou a qualquer outra fonte, TODA a submissão será descartada.

Orquestrador de Processos

Um orquestrador de processos, chamado ProcessFlow, deverá ser implementado nesta atividade. O programa receberá tarefas que representam programas existentes no sistema e será responsável por criá-las e executá-las por meio de processos filhos. O ProcessFlow deve funcionar de duas formas: interativo ou workflow. No modo interativo, deve-se apresentar o prompt `processflow>` para que a/o usuário/a digite os comandos. No modo workflow, o programa recebe um arquivo `.pf` como argumento, contendo uma lista de comandos a serem executados. Nesse modo, o prompt não será apresentado; no entanto, cada linha lida do arquivo deve ser impressa antes de ser processada. Em ambos os modos, o programa será finalizado com o comando `exit`.

Antes de executar uma tarefa, ela deverá ser cadastrada utilizando:

```
processflow> task <nome> <programa> [argumentos...]
```

Por exemplo:

```
processflow> task listar /bin/ls -l
```

A tarefa cadastrada poderá ser executada com `run <nome>`. Mais de uma tarefa poderá ser executada de forma sequencial ou paralela. No modo sequencial, uma tarefa somente deve iniciar quando a anterior terminar. No modo paralelo, todas devem ser iniciadas antes que o ProcessFlow espere pelo término do grupo.

```
processflow> run sequential tarefa1 tarefa2 tarefa3
```

```
processflow> run parallel tarefa1 tarefa2 tarefa3
```

O ProcessFlow também deve dar suporte a:

1. Pipe - A saída de uma tarefa poderá ser enviada como entrada de outra tarefa. Por exemplo:

```
processflow> run pipe listar ordenar contar
```

em que `listar`, `ordenar` e `contar` são tarefas previamente cadastradas e cada uma deve executar em um processo diferente.

2. Redirecionamento de entrada e saída - Uma tarefa poderá receber sua entrada de um arquivo ou enviar sua saída para um arquivo. Exemplos:

```
processflow> input ordenar nomes.txt
```

```
processflow> output ordenar resultado.txt
```

```
processflow> append ordenar historico.txt
```

3. Diretório de trabalho - O comando `workdir <diretório>` deverá alterar o diretório utilizado pelas tarefas executadas posteriormente.

4. Execução em background - O comando `start <tarefa>` deverá iniciar uma tarefa e disponibilizar novamente o prompt sem esperar seu término. O ProcessFlow deverá informar um identificador do job e o PID do processo.

```
processflow> start processamento
```

```
[1] 1234
```

O comando `jobs` deverá listar os jobs iniciados em background e o comando `wait <jobid>` deverá permitir aguardar o término de um job específico. O ProcessFlow é responsável por coletar corretamente o término dos processos filhos que criar.

Algumas especificações

1. O programa deve ser implementado em C e, para execução das tarefas, deve-se usar `fork()`, `exec()`, `wait()/waitpid()`, `dup2()` e `pipe()`. O ProcessFlow será o processo pai responsável por criar e gerenciar seus processos filhos. Não será permitido utilizar `system()`, `popen()` ou delegar a execução para outro shell.

2. Deve ser executável em sistemas Linux, Unix ou macOS.

3. A execução deve ser realizada utilizando o comando `./processflow [workflowFile]`, em que `workflowFile` é opcional. Se o argumento não estiver presente, o programa deverá executar no modo interativo.

O programa não deve falhar ao encontrar um erro. É necessário checar os parâmetros antes de aceitá-los. Assim, o ProcessFlow deve apresentar uma mensagem de erro coerente e continuar o processamento ou encerrar a execução, a depender das situações descritas abaixo:

1. Imprimir uma mensagem e encerrar a execução quando:

- Um número incorreto de argumentos for passado ao iniciar o ProcessFlow;
- O arquivo workflow não existir ou não puder ser aberto.

2. Imprimir uma mensagem de erro e continuar processando quando:

- Uma tarefa informada não existir;
- O programa associado à tarefa não existir ou não puder ser executado;
- Um arquivo de entrada ou saída não puder ser aberto;
- Um job informado não existir;
- Um diretório informado em workdir não existir.

Algumas outras situações que, apesar de não serem erros, o ProcessFlow deve tratar de forma coerente:

- Linha de comando vazia no prompt;
- Múltiplos espaços em branco em uma linha;
- workflowFile sem o comando exit e CTRL-D no modo interativo;
- Processos que terminam com código de saída diferente de zero;
- Processos em paralelo terminando em ordens diferentes.

O que deve ser submetido pelo Classroom:

1. Relatório em pdf, descrevendo o que foi feito e como a implementação foi realizada, com evidências dos testes executados. Apresente as principais dificuldades, limitações e o que não conseguiu implementar, se houver. O formato deverá seguir o Guia de Relatórios da disciplina.
2. Os arquivos com o código-fonte..
3. Um breve README descrevendo os arquivos utilizados, como executar, compilar e testar o programa, além de especificar em que sistema operacional foi implementado.
4. Incluir o Makefile para compilação, limpeza dos arquivos compilados e execução dos testes, se houver.
5. Incluir o link do Github, faça commits atômicos para se adequar ao relatório.

Rubricas

Item	Peso
README + Makefile	10%
Relatório	20%
Execução simples + exit	10%
Sequencial + paralelo	10%
Pipe	10%
Redirecionamento	10%
Background + jobs + wait	10%
workflowFile + workdir	10%
Erros + outras situações	10%